

DOC-04

Logical & Technical Architecture

System design, request lifecycle, and architectural decisions

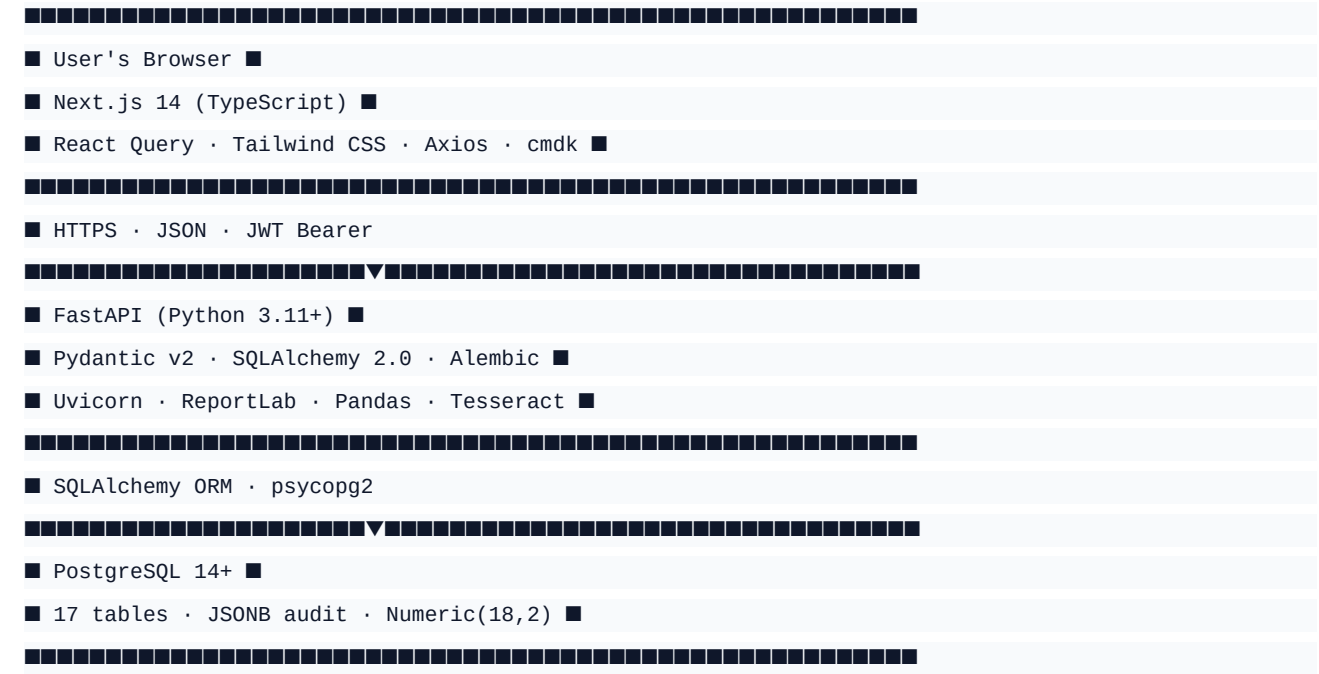


Project	GPA-ERP V5.0
Type	Construction Cost Control ERP
Stack	FastAPI · Next.js 14 · PostgreSQL
Version	V5.0

GPA-ERP — Logical & Technical Architecture

System Overview

GPA-ERP is a **construction cost control ERP** built as a two-tier web application:



Technology Stack

Backend

Component	Technology	Version
Web Framework	FastAPI	0.115.5
Language	Python	3.11+
ORM	SQLAlchemy	2.0.36
Migrations	Alembic	1.14.0
Auth	python-jose + passlib[bcrypt]	—
Validation	Pydantic	v2 (2.10.3)
PDF Generation	ReportLab + pypdf	4.2.5 / 4.3.1
Data Import	Pandas + OpenPyxl	2.2.3 / 3.1.5
ASGI Server	Uvicorn	0.32.1
Database Driver	psycopg2-binary	—

Frontend

Component	Technology	Version
Framework	Next.js (App Router)	14.2.18
Language	TypeScript	5.7.2
UI Library	React	18.3.1
Styling	Tailwind CSS	3.4.16
HTTP Client	Axios	1.7.9
Server State	TanStack React Query	5.62.7
Forms	React Hook Form + Zod	7.54.2 / 3.23.8
Rich Text	TipTap	3.23.1
Charts	Recharts	2.13.3
OCR	Tesseract.js	7.0.0
Icons	Lucide React	—

Request Lifecycle

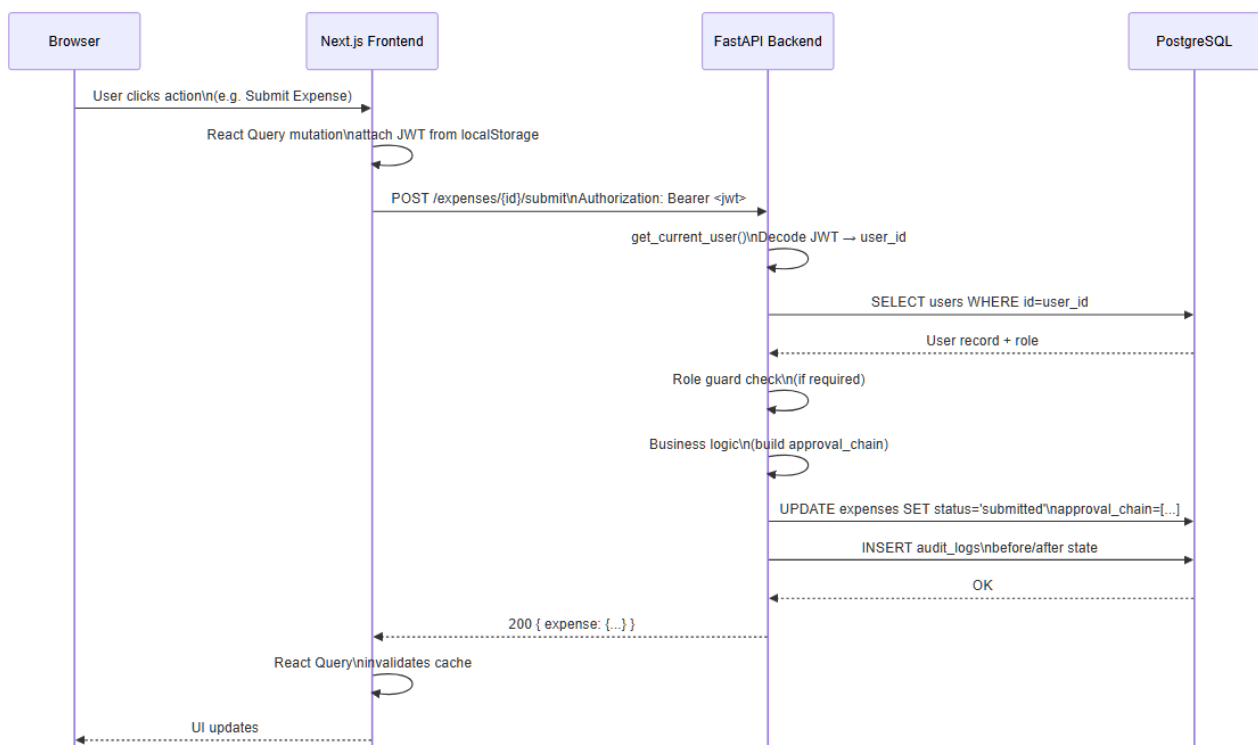


Figure 1

Authentication & Authorization

JWT Flow

1. POST /auth/login { email, password }
2. Backend hashes password → compares with bcrypt hash in DB
3. Backend issues JWT: { sub: user_id, exp: now+480min }
4. Frontend stores token in localStorage as "gpa_token"
5. All subsequent requests: Authorization: Bearer <token>
6. On 401: frontend clears token, redirects to /login

Token configuration:

- Algorithm: HS256
- Expiry: 480 minutes (8 hours)
- Secret: SECRET_KEY env var (64+ chars recommended)

Role Guards (Backend)

```
# Layer 1 – Auth only (any logged-in user)

async def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) -> User

# Layer 2 – Role-specific

def require_role(*roles: RoleName):
    def guard(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role.name not in roles:
            raise HTTPException(403, "Insufficient permissions")
        return current_user
```

Menu-Based Access Control

A second authorization layer allows SUPER_ADMIN to grant or revoke specific menu access per user via UserMenuPermission. This supplements (not replaces) role-based guards.

Revenue-Driven Budget Model

The core financial constraint of the system: **you cannot spend more than you have earned.**

```
# Project hybrid properties (computed at DB level via scalar subqueries)

project.total_revenue = SUM(actual_payment ?? amount WHERE status='confirmed')
project.total_committed = SUM(amount WHERE status IN (verified, approved, paid, hard_locked))
project.budget = total_revenue - total_committed
```

This means:

- **Draft** ARs do not count as revenue (unconfirmed billing)
- **Draft/submitted** expenses do not count as committed (not yet verified)
- **Rejected** expenses have zero budget impact
- The budget is a **live calculation** — no separate budget table

Approval Matrix Engine

The approval chain is configurable at runtime via the approval_rules table (no code changes required):

```
# On expense submit:
rules = db.query(ApprovalRule).filter(
ApprovalRule.is_active == True,
ApprovalRule.min_amount <= expense.amount,
or_(ApprovalRule.max_amount == None, ApprovalRule.max_amount > expense.amount),
or_(ApprovalRule.cost_code_category == None,
ApprovalRule.cost_code_category == cost_code.category)
).order_by(ApprovalRule.priority).all()

chain = [rule.required_role for rule in rules] # e.g. ["COST_CONTROL", "PM", "MD"]
expense.approval_chain = chain
expense.approval_step = 0
expense.current_approver_role = chain[0] if chain else None
```

The chain is stored as **immutable JSON** on the expense record — changing the rules table does not retroactively affect in-flight expenses.

Audit Trail Design

```
# audit.py – immutable append-only log

def log_action(db, entity_type, entity_id, action, before, after, user_id, ip):
    log = AuditLog(
        entity_type=entity_type,
        entity_id=entity_id,
        action=action,
        before_state=before, # JSONB snapshot
        after_state=after, # JSONB snapshot
        changed_by=user_id,
        ip_address=ip,
    )
    db.add(log)
    db.commit()

# Never update or delete AuditLog rows
```

The `before_state` and `after_state` fields store a full JSON snapshot of the record before and after the change. This allows **point-in-time reconstruction** of any entity.

File Structure

```
gpa-erp/ # Backend
├── app/
│   ├── main.py # FastAPI app, CORS, router registration, lifespan
│   ├── config.py # Pydantic Settings from .env
│   ├── database.py # SQLAlchemy engine + SessionLocal
│   └── models.py # 17 ORM models + enums
```

```
■ ■■■ schemas.py # Pydantic v2 request/response models
■ ■■■ dependencies.py # JWT auth, role guards, approval matrix builder
■ ■■■ audit.py # Immutable audit log service
■ ■■■ menu_permissions.py # Role-based menu defaults
■ ■■■ pdf_generator.py # Legal document PDF (KOP SURAT)
■ ■■■ routers/
■ ■■■ auth.py # POST /login, GET /me
■ ■■■ users.py # User CRUD
■ ■■■ projects.py # Projects + Excel import
■ ■■■ receivables.py # AR / Revenue
■ ■■■ expenses.py # Full approval workflow
■ ■■■ petty_cash.py # Petty cash reports
■ ■■■ legal.py # Legal documents + signatures
■ ■■■ inventory.py # Stock management
■ ■■■ vault.py # Cost codes, approval rules, audit log
■ ■■■ admin.py # Backend HTML admin panel
■■■ alembic/ # Database migrations
■■■ scripts/ # Seed & import scripts
... (27 more lines)
```

Environment Configuration

Variable	Default	Required
DATABASE_URL	postgresql://gpa_user:gpa_passwords@localhost:5432/gpa_erp	Yes
SECRET_KEY	—	Yes (64+ chars)
ALGORITHM	HS256	No
ACCESS_TOKEN_EXPIRE_MINUTES	480	No
ALLOWED_ORIGINS	http://localhost:3000	Yes (prod: set frontend URL)
DEBUG	false	No
UPLOAD_DIR	./uploads	No
MAX_UPLOAD_MB	10	No
SEED_SUPER_ADMIN_EMAIL	admin@gpa.local	Seed only
SEED_SUPER_ADMIN_PASSWORD	ChangeMe123!	Seed only

Database Indexes

Critical indexes for query performance:

Table	Index	Purpose
users	email	Login lookup
expenses	project_id + status	Per-project expense lists
expenses	current_approver_role	Action center queue
expenses	submitted_by	My expenses filter
account_receivables	project_id + status	Per-project AR
account_receivables	invoice_no	Invoice lookup
audit_logs	entity_type + entity_id	Per-entity audit history
audit_logs	created_at	Time-range audit queries
petty_cash_reports	project_id + month	Monthly petty cash

Known Architecture Decisions & Trade-offs

Decision	Reason	Trade-off
JWT in localStorage	Simple implementation, no cookie config needed	XSS risk — mitigated by CSP headers
Budget as hybrid property	Always accurate, no sync issues	Computed on every read — may need caching at scale
Approval chain stored as JSONB	Rule changes don't affect in-flight expenses	Harder to query "all expenses needing PM approval" (requires JSONB contains)
Single PostgreSQL DB	Simpler ops, strong consistency	Not horizontally scalable — acceptable for ERP scale
Server-rendered admin panel	No JS dependency, secure fallback	Separate auth system from frontend JWT
Client-side search (legacy)	Fast for small datasets	Fetches all records — being migrated to server-side (Phase 3/5)